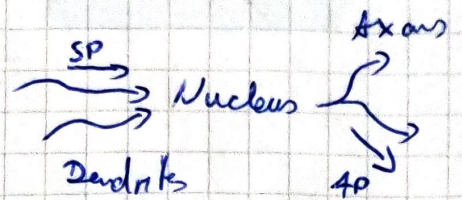


43

Computational Neural Networks



Signal transmission in the brain:

- Wave of voltage change along an axon
- Nucleus generates an Action Potential (AP) if the sum of its incoming Synaptic Potentials (SP) is strong enough

Human Brain:

- Complex physical and chemical activity to transmit a single action potential along one connection

Artificial Neural Networks:

- All physics and chemistry represented by a few parameters associated with nodes and axes

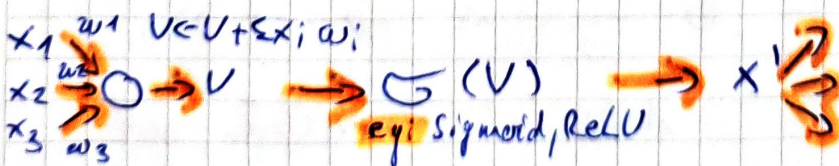
Biological → Artificial NN's:

→ Structural Abstraction

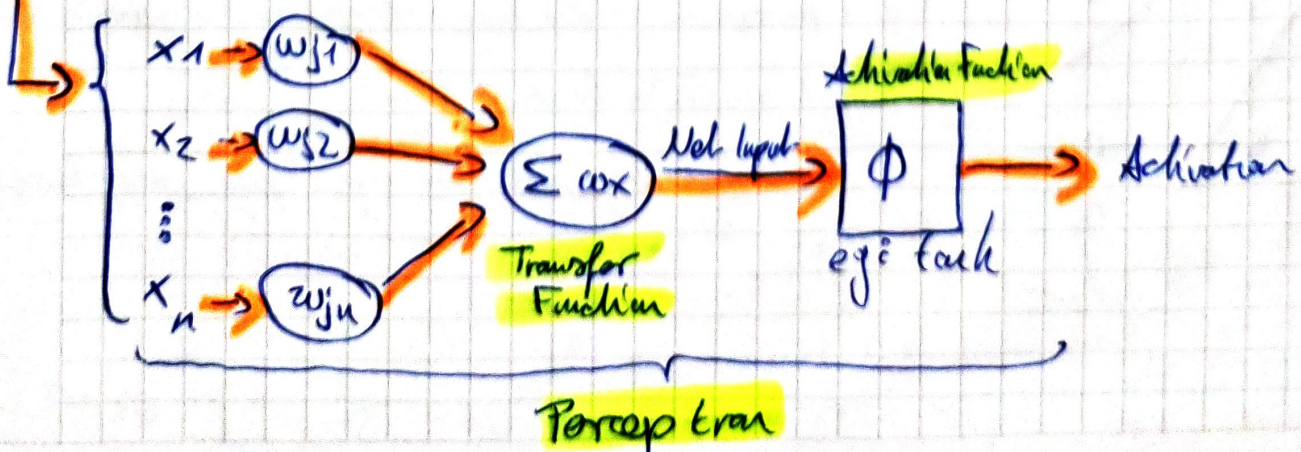
→ Temporal abstraction
(Voltage to activation values)

→ Functional Abstraction
(activation functions)

Reset and Activation Functions:



Reset: → never reset V
→ reset V when above threshold
→ Always reset V



23

$$\vec{x} \rightarrow \begin{bmatrix} l_1 \end{bmatrix} \rightarrow \begin{bmatrix} l_2 \end{bmatrix} \rightarrow \begin{bmatrix} l_3 \end{bmatrix} \rightarrow \phi(\vec{x}) = \vec{y}$$

The weights in a NN determine the approximated function!

eg: Image Classification

How does the perceptron learn?

→ Update network weights based on an existing set of input-output pairs $(\phi(\vec{x}), \vec{y})$

eg: linear activation function

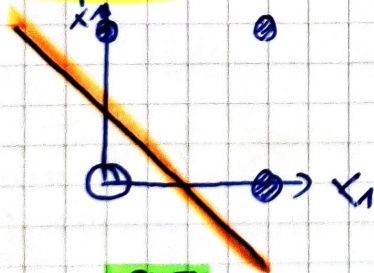
Desired Output : $t \Rightarrow \text{Error} = t - y$

→ Learning Rule : $\Delta w_i = \eta (t - y) x_i$, $\eta = \frac{1}{35}$

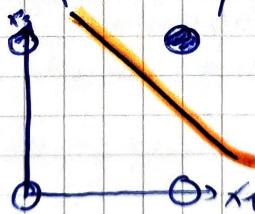
Perceptron : Convergence Theorem

→ Given linearly separable data, the perceptron will find a separating hyperplane in a finite amount of steps

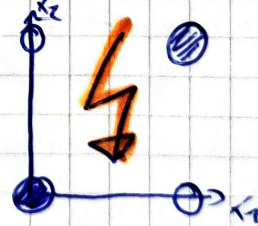
Perceptron : Limitations in learning Boolean functions



OR

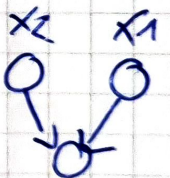


AND

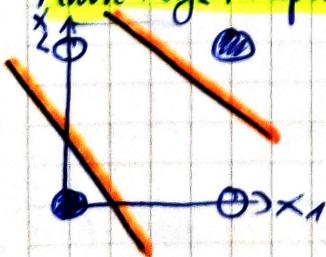


XOR

not

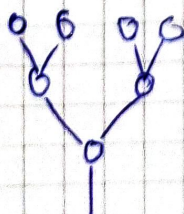


Multi-Layer Perceptron : Boolean Functions



Each perceptron in the lower layer can separate linearly
Output neurons in the last layer:

→ can separate the input space non-linearly



L3

• Deep NN training algorithm:

→ Randomly initialise connection weights

→ Start loop:

→ For each training pair of inputs $\vec{x}$ and output targets $\vec{t}$:

↳ Forward Pass: compute $\phi(\vec{x}) = \vec{y}$ performing matrix multiplications for each layer

↳ Update: weights based on $\vec{y}$ and $\vec{t}$ ← should be value

→ If average difference between $\vec{y}$ and $\vec{t}$ is small enough:

↳ end loop and return weights

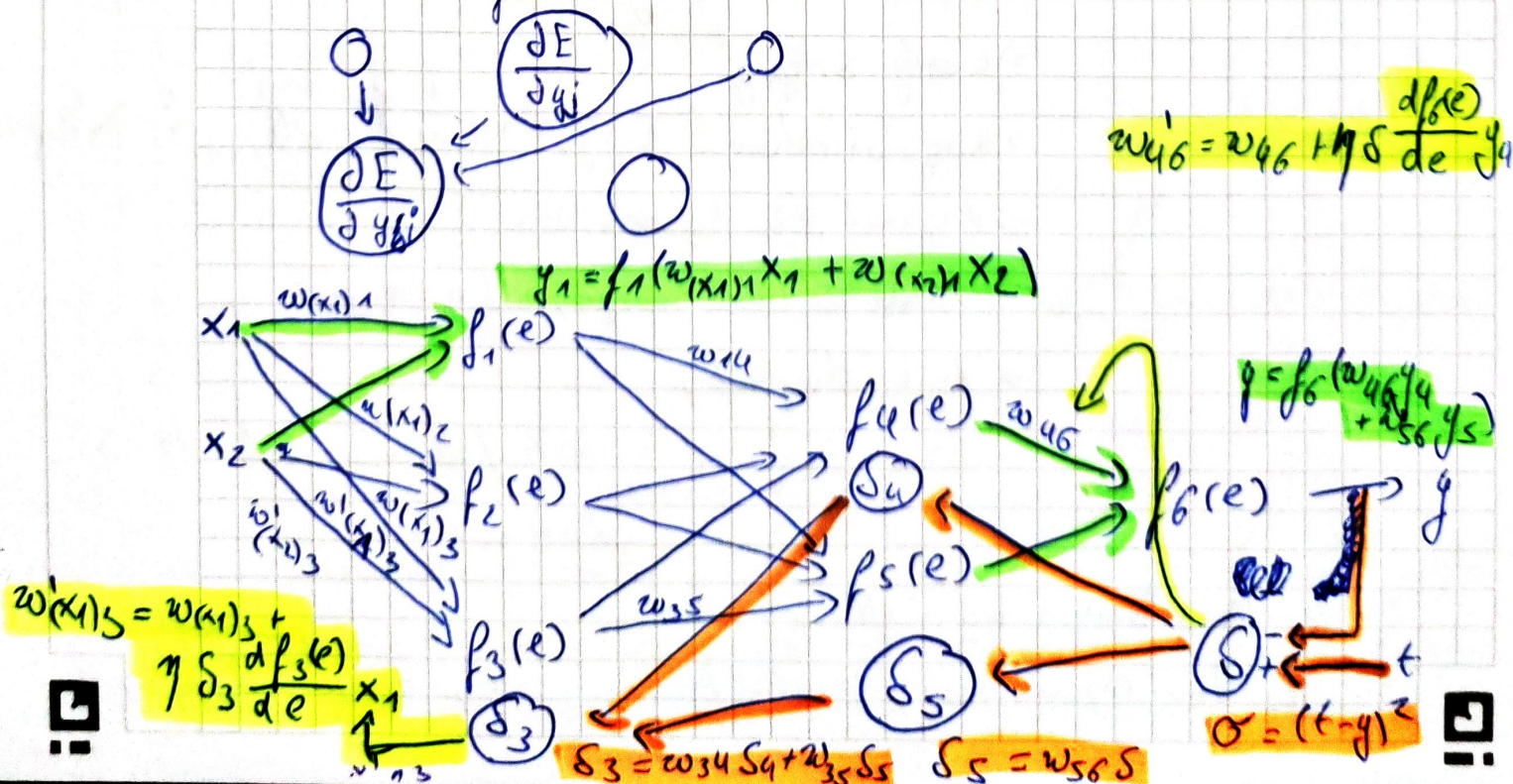
• Backpropagation:

→ For a training pair $\vec{x}, \vec{t}$ and a network output $\vec{y}$:

1) Compute $E(\vec{y}, \vec{t})$ eg: $\frac{1}{N} \sum_{i \in N} (t_i - y_i)^2$ (Mean Squared Error)

2) Compute error ~~derivation~~ derivatives in each hidden layer from error derivatives in subsequent layer $\frac{\partial E}{\partial y_j} = t_j - y_j$

3) Use error derivatives for activations to get error derivatives for the weights



23

• Training Issues:

→ How to update the weights?

→ After each training sample: Incremental / Online Training

→ At the end of each epoch:

→ Batch training: Epoch ends when all training samples seen

→ Mini-Batch training: Epoch ends when subset of training data seen

→ Learning Parameters: learning rate, momentum

• Generalization Problem:

~ Overfitting (adapting to noise)

— Underfitting (little or no adaptation to structure of data)

• Overfitting:

→ training on small training set $\Rightarrow$ poor performance

→ Instead of generalizing, the network learns the particularities of individual samples

→ Solution:

→ Generate more training data

→ Early stopping

→ Regularization: $E(\vec{y}, \vec{\epsilon}) = \frac{1}{n} \sum_{i \in N} (b_i - y_i)^2 + \lambda \sum_j w_j$

→ Augment the dataset:

→ add distorted images, add noise, etc

→ Apply dropout:

→ Randomly set $\approx 50\%$ of all neuron activations to 0 during training

• Weights & Biases as a library:

Real time experiment tracking, Visualizing the training process, Analyze model performance

L3

Recurrent Neural Networks

→ ^{How} NN perform inference on variable-length sequences with temporal context

→ Recurrent Network with cycles

↳ Network has internal state: It can remember past states

↳ Natural modeling of sequential data:

The network can operate over sequences of vectors, not just fixed-length windows

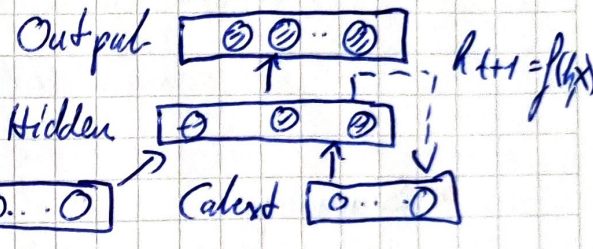
↳ It can behave chaotically or oscillate (eg: good for weather)

Simple Recurrent Network (SRN)

→ Backpropagation realized by unrolling recurrent connections, i.e. backpropagation through time

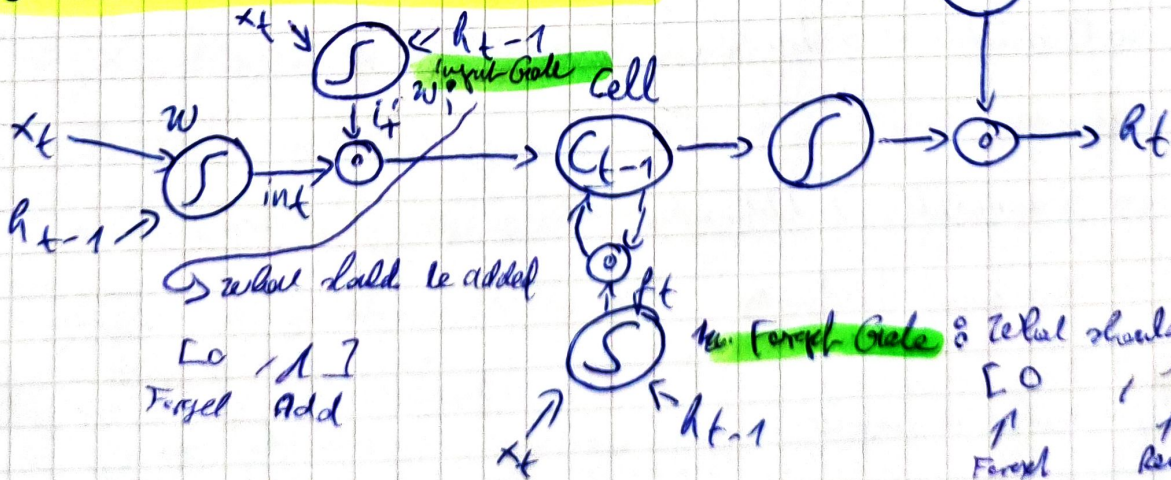
→ Biologically more plausible than MLP

→ Prone to vanishing gradient problem



→ gradient becomes small
↳ not update

Gated recurrent neural networks



→ what should be added

Lo [1] Forget Add

→ Forget Gate: what should be forgotten
Lo [0] Forget Remainder

→ long short term memory (LSTM)

→ addresses vanishing gradient problem

→ update gate and reset gate ⇒ what information should passed to the output

L3

Learning Paradigms

Supervised :

- Detailed feedback that includes the correct response to each input ; omnipresent teacher , eg: turn left

Unsupervised :

- No feedback ; ANN tries to intelligently cluster inputs and learn proper correlations between components of input space

After a long consider
eg: a left

Reinforced :

- Simple feedback mainly at the end of a problem-solving attempt
eg: you are at the goal

Summary

ANN some aspects of biological neural structures

- Nodes / units and weighted connections $\leftrightarrow$ neurons and synapses
- Activation values $\leftrightarrow$ action potentials
- Backpropagation $\leftrightarrow$ plasticity
- Reward $\leftrightarrow$ dopamine

Structures

- Convolutional Networks
- Recurrent Networks

eg:

Foundation models, Speech processing, Cross-modal lang

Autoencoders :

Error based on reconstruction quality

